Des agents IA aux données fiables

HydroDataLab
Une pratique concrète pour les analyses Python

20 octobre 2026 • Sources vérifiées le 18 octobre • Démonstration pédagogique, données publiques

Votre terrain de développement

Analystes BI et développeurs citoyens : Python, ETL et visualisation de données.

Développer

Explorer une idée.
Créer un traitement ou un visuel.

→

Tester sur le poste

Vérifier les calculs.
Rejouer avec des données d'essai.

→

Préparer le déploiement

Emballer code et dépendances pour un environnement conteneurisé.

Une marge pour expérimenter

Un cadre plus souple : l'occasion d'essayer, de comparer et d'apprendre.

La sécurité au quotidien

Données autorisées, secrets protégés.
Accès limités, code relu et testé.

> L'IA accélère les essais; vous gardez la maîtrise des données et des actions.

Microsoft · sécurité du développement avec les agents

Du besoin BI au résultat vérifié

Notre défi : détecter les mesures manquantes dans une journée de données.

Ce que vous saurez faire

Donner une tâche précise à l’agent.
Retrouver les règles qu’il utilise.
Vérifier le calcul qu’il propose.

Notre exemple Python

Données publiques Hydro-Québec.
Un traitement réutilisable.
Un indicateur de qualité visible.

> 45 minutes de présentation et de démos, puis vos questions.

Quatre séries, quatre sens différents

Série	Unité	Ce que l’on observe
Demande électrique au Québec	MW	Une puissance demandée
Niveau d’eau au réservoir Gouin	m	Une hauteur dans le référentiel source
Température minimale	°C	Un minimum sur la période source
Température maximale	°C	Un maximum sur la période source

Notre ETL : extraire les données, les transformer, puis charger ou exporter le résultat.
Un niveau en mètres ne donne pas une réserve en GWh.

Sources publiques HQ et contrat [S11, S12]. Instantané : 17 octobre 2026, 168 observations; récupéré le 18 octobre à 19:44 UTC.

Un modèle, un harnais, un agent

Notion	Dans notre tâche Python
Modèle	Propose du code et raisonne sur le résultat.
Outil · tool	Lit un fichier, le modifie ou lance Python.
Harnais · harness	Relie modèle, contexte, outils et autorisations.
Agent	Boucle sur ces actions pour accomplir votre tâche.

> **Notre choix : harnais Copilot dans VS Code.**
> Il utilise le Copilot SDK. Choisir un modèle Claude conserve ce harnais.

Microsoft · comprendre les harnais · Nouveautés 1.142

Git : garder une trace de ses essais

Mot rencontré	Ce qu'il signifie
Dépôt Git	Le projet et l'historique de ses versions enregistrées.
Commit	Un point enregistré dans cet historique.
Branche	Une suite de versions pour un essai, par exemple `demo-live`.
Worktree · facultatif	Un autre dossier du même dépôt, pour travailler sur une autre branche.

> Pour débuter, une branche d'essai suffit.

Documentation Git · worktree [S22]. Les fichiers et résultats non enregistrés demandent leur propre sauvegarde.

Démo 1 · donner une mission à l'agent

> « Repère le calcul des indicateurs et explique comment il est testé.
> Ne modifie aucun fichier. »

Repère dans la fenêtre Agents	À quoi il sert
Session	Garder une conversation consacrée à cette tâche.
Planification	Demander une démarche avec `/plan` avant de modifier.
Autorisations	Choisir quelles actions doivent être confirmées.

Quelles sources permettent de vérifier sa réponse?

Microsoft · fenêtre Agents · Planification · Réglages

Écrire une règle que l’on peut vérifier

Exemple fictif : trois créneaux horaires de température attendus.

08 h

10 °C

Mesure reçue

09 h

0 °C

Mesure reçue

10 h

None

Aucune mesure

> **2 mesures valides sur 3 : couverture de 66,67 %.**
> Remplacer l’absence par 0 inventerait une troisième mesure.

Instruction permanente : « Conserver les valeurs manquantes.
Compter 0 comme une mesure numérique valide. »

`.github/instructions/python.instructions.md`

Microsoft · instructions personnalisées · Voir le panneau Instructions →

Où écrire la règle, et quand s'applique-t-elle?

Socle Copilot : `.github/copilot-instructions.md` · Markdown commun au projet.
Règle ciblée : `.github/instructions/python.instructions.md`

```
---
description: Qualité des transformations Python
applyTo: "etl/**/*.py"
---
Conserver les valeurs manquantes.
Compter 0 comme une mesure numérique valide.
```

applyTo : motif des fichiers créés ou modifiés. **description** : pertinence de la tâche.
Sans ces deux champs, joindre le fichier manuellement.

> Pour ce groupe : socle Copilot + règles Python ciblées.
> Plusieurs harnais compatibles? Préférer `AGENTS.md` pour le socle partagé.

Microsoft · format et activation · AGENTS.md

Sept concepts, sept responsabilités

Concept	Question à laquelle il répond	Petit exemple BI
Instructions	Quelles règles respecter?	Garder les valeurs manquantes.
Skill	Quelle procédure réutiliser?	Tests → unités → rapport qualité.
Agent personnalisé	Quel rôle et quels outils?	Réviseur BI en lecture seule.
Tool · outil	Quelle action exécuter?	Lire un CSV ou lancer pytest.
Serveur MCP	Comment connecter un système?	Fournir un outil de lecture SQL.
Hook	Que lancer à un événement?	Journaliser un appel d'outil.
Plugin	Comment distribuer un ensemble?	Installer une skill et un serveur MCP.

L'instruction guide le modèle; le hook exécute du code à l'événement configuré.
Le serveur MCP expose des outils; le plugin peut les regrouper avec d'autres capacités.

Microsoft · comparer les concepts · Skills · Hooks · Plugins

Context layers · les couches du contexte

Des consignes durables → une demande précise → des résultats observés.



Couches complémentaires, sans priorité imposée. Organisation pédagogique du cours, non officielle.

Microsoft · activation des règles · Structurer le contexte

Les tokens : lire et produire

Un token est un morceau de texte :
un mot, une partie de mot ou un signe.

Ce que le modèle reçoit

Votre demande et les consignes.
Les extraits de code ou de données.
Les résultats des outils.

Ce que le modèle produit

Sa réponse et le code proposé.
Du raisonnement facturé
selon le modèle utilisé.

> Pour notre ETL : le schéma, un petit échantillon représentatif
> et les tests donnent un contexte utile.

Microsoft · contexte · GitHub · facturation. Évaluer toutes les tentatives nécessaires.

Auto : un bon point de départ

Niveau	Priorité	Exemple BI
Efficiency	Coût	Expliquer une fonction courte
Balance	Coût, qualité et temps d’attente	Transformer une série avec tests
Intelligence	Qualité	Investiguer une anomalie difficile

> Auto choisit selon la tâche et les modèles autorisés.

Plans payants : remise Auto de 10 % sur le coût des modèles.
Le volume total dépend du travail effectué.

GitHub · Auto model selection, consulté le 18 octobre 2026. Politiques et abonnement restent déterminants.

Comparer 4 tâches et 5 modèles

Auto Balance au quotidien · choix manuels ci-dessous

− 1 essai(s) +

Modèle Tokens entrée / sortie →	**Simple** Expliquer du code 3 000 / 500	**Délimitée** Modifier un ETL 20 000 / 2 000	**Complexe** Plusieurs fichiers 60 000 / 8 000	**Diagnostic difficile** Trouver une cause 60 000 / 8 000
GPT-6 Luna	◆ 0,00055	0,003	0,010	0,010
Claude Haiku 5.5	◆ 0,00055	0,003	0,010	0,010
Claude Sonnet 5.5	0,011	◆ 0,060	0,200	0,200
GPT-6.1 Sol	0,011	0,060	◆ 0,200	0,200
Claude Opus 5.5	0,022	0,120	0,400	◆ 0,400

◆ Choix conseillé à évaluer

Échelle commune des barres : 0 → 0,400 $ US

Coûts simulés en $ US. Volumes fictifs par essai, identiques entre modèles pour chaque tâche.
Sans cache ni remise Auto. Comparer le coût total d'un résultat validé; ce n'est pas un benchmark.

GitHub · tarifs au 18 octobre 2026 · Comparer les modèles

Auto choisit; HydraFusion orchestre

Auto

Un modèle choisi pour la requête.
Préférence Efficiency, Balance ou Intelligence.

HydraFusion · préversion

Single : un modèle.
Cascade : proposition → jugement.
Critique : proposition → revue → révision.

> **Coût HydraFusion = somme des modèles utilisés.**
> Pas de supplément propre à HydraFusion; pas de remise Auto.
> Max augmente l'effort sur les tâches les plus exigeantes.

GitHub · fonctionnement et facturation HydraFusion
Annonce du 14 octobre · Documentation Auto

Démo 2 · suivre le trajet des données

Données HQ → **Calcul Python** → **Tableau de bord**

Élément préparé	Son rôle
API .NET	Transmettre les commandes et les résultats à l'interface Vue.
Aspire	Démarrer les composants et regrouper leurs journaux.
Image Docker	Emballer le code et ses dépendances pour l'exécuter en conteneur.

> D'où vient ce résultat? Et comment reprendre un traitement en erreur?

Architecture HydroDataLab [S12]. Le calcul métier reste dans le même code Python.

SDD : une spécification avant le code

Spec-Driven Development · développement piloté par spécifications.
Le besoin devient des critères, puis un plan, du code et des preuves.
Notre règle : la **couverture** mesure la part des créneaux attendus avec une mesure valide.

Exemple sur une série horaire	Critère de réussite
24 mesures valides sur 24 attendues	100 %
23 mesures valides sur 24 attendues	95,83 %
Valeur zéro, doublon ou absence	Zéro compte; doublon et absence n'ajoutent rien.
Changement d'heure	23 ou 25 heures attendues selon la journée.

> Ces exemples deviennent des tests. Le SDD sera approfondi dans la prochaine formation animée par l'autre équipe.

GitHub · Spec Kit et développement piloté par spécifications

Démo 3 · détecter une mesure absente

24 mesures horaires attendues.
23 mesures numériques valides.

23 ÷ 24 = 95,83 %

> L’agent ajoute le calcul en Python.
> Nous vérifions les tests et l’indicateur affiché.

Scénario pédagogique : une mesure est retirée en mémoire. Les données brutes restent intactes.

Votre prochaine tâche Python

Écrivez un résultat attendu, confiez une petite tâche, puis examinez les preuves.

Apprendre et échanger

Microsoft Learn : agents
Pratiques VS Code
Awesome Copilot
GitHub Community

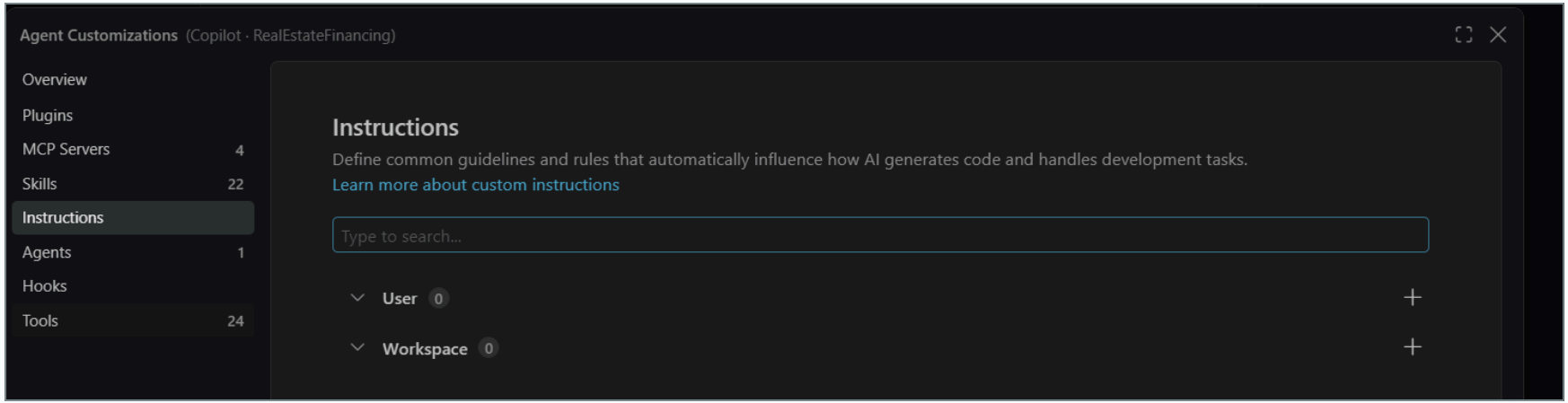
À découvrir : Canvas

Une surface interactive partagée avec l'agent, à côté du chat. Exemple : suivre les étapes de l'ETL dans un tableau de tâches.

Microsoft · Canvas en préversion
Voir le fonctionnement →

> Quelle règle de vos analyses aimeriez-vous rendre vérifiable?

Annexe · du fichier au panneau Instructions



User · vos préférences

`~/.copilot/copilot-instructions.md`

Commun à vos projets, sur ce poste.

Workspace · ce projet

`.github/copilot-instructions.md`

Partagé dans Git avec l'équipe.

Règles ciblées : `~/.copilot/instructions/` ou `.github/instructions/` .

Microsoft · panneau Customizations · Retour à l'exemple →

Capture réelle fournie par le présentateur, autre projet, listes vides. Les chemins ci-dessus expliquent les portées; ils ne sont pas des entrées visibles dans cette capture.

Annexe · modèles et tarifs Copilot

Modèle	Entrée	Cache lu	Cache écrit	Sortie
GPT-6 Luna	0,10	0,01	0,125	0,50
Claude Haiku 5.5	0,10	0,01	0,125	0,50
GPT-6.1 Sol	2,00	0,10	2,50	10,00
Claude Sonnet 5.5	2,00	0,10	2,50	10,00
Claude Opus 5.5	4,00	0,20	5,00	20,00

$ US / million de tokens, tarifs standards. 1 AI credit = 0,01 $ US.
Long contexte : Haiku > 100k; Luna et Sol > 272k → grille distincte.

GitHub · grille officielle des tarifs, 18 octobre 2026. Disponibilité HQ à confirmer.

Annexe · Canvas, travailler dans le même objet



Schéma pédagogique : une extension fournit l'interface et ses actions.
Ce dessin et le tableau de bord Vue ne sont pas des Canvas VS Code.

Ouvrir une surface

Activer `chat.canvases.enabled`.
Créer une nouvelle session Copilot.
`/create-canvas` si disponible;
puis **+** → **Canvas** pour un canvas installé.

Cadre actuel · Preview

Application VS Code de bureau.
Session locale Copilot / Agent Host.
Pas le harnais « Local », ni Claude,
Codex ou une session distante.

Microsoft · prérequis et fonctionnement · Nouveauté 1.142 · Retour →

Annexe · le contrat d'une observation

Dimension	Question à résoudre
Schéma et identité	Quelle série, station et instant unique?
Unité et sens	MW, m ou °C? Mesure, minimum, maximum?
Fréquence	15 minutes, horaire ou quotidien?
Temps	Instant UTC; journée America/Toronto?
Provenance	URL, licence, récupération et version brute?
Qualité	Null, doublon, invalidité et couverture attendue?

Contrat HydroDataLab [S12] et métadonnées publiques HQ [S11]. Préserver le sens de la source.

Annexe · reprendre sans improviser

00 socle → **01 instructions** → **02 extraction** → **03 BI** → **04 pilotage** → **05 Docker** → **06 spec** → **07 solution**

- Un tag Git est un repère nommé sur une version enregistrée.
- Conserver ses modifications avant de changer de version.
- Séparer les données d’exécution des fichiers de code.
- Reprendre avec la copie locale des données et les tests.

> PDF, solution préparée et Git pour reprendre.

Guide présentateur et procédure de reprise du projet. Les checkpoints VS Code n’annulent pas les effets externes des outils [S06].

Annexe · sources et limites

VS Code et GitHub

Personnalisations
Contexte
Auto
Tarifs Copilot
HydraFusion, 14 octobre

Données et pratique

Données ouvertes HQ
Microsoft Learn
Awesome Copilot
Spec Kit

> Sources consultées le 18 octobre 2026. Les fonctions disponibles dépendent de la version et du compte.

Registre complet : docs/SOURCES.md. Schémas pédagogiques; aucune capture VS Code inventée.